```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int size = 0;
struct symbtab {
    char label[10];
    int addr;
    struct symbtab *next;
};
struct symbtab *first = NULL, *last = NULL;
void insert();
void del();
int search(char lab[]);
void modify();
void display();
int main() {
    int op;
    char la[10];
    do {
        printf("\n--- SYMBOL TABLE IMPLEMENTATION ---");
        printf("\n1. INSERT\n2. DISPLAY\n3. DELETE\n4. SEARCH\n5. MODIFY\n6. END\n");
        printf("Enter your option: ");
        scanf("%d", &op);
        switch (op) {
            case 1:
                insert();
                display();
                break;
            case 2:
                display();
                break;
            case 3:
                del();
                display();
                break;
            case 4:
                printf("Enter the label to be searched: ");
                scanf("%s", la);
                // The address display is now handled inside search()
                if (search(la) == 0) {
                    printf("The label is NOT found in the symbol table.\n");
                }
                break;
            case 5:
```

```
            modify();
            display();
            break;
        case 6:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid option!\n");
    }
  } while (op != 6);
  return 0;
}
// --- UPDATED SEARCH FUNCTION ---
int search(char lab[]) {
  struct symbtab *p = first;
  while (p != NULL) {
    if (strcmp(p->label, lab) == 0) {
        printf("\nLabel '%s' found!", p->label);
        printf("\nAddress: %d\n", p->addr);
        return 1; // Success
    }
    p = p->next;
  }
  return 0;
}
void insert() {
  char l[10];
  printf("Enter the label: ");
  scanf("%s", l);
  struct symbtab *p_check = first;
  int exists = 0;
  while(p_check != NULL) {
    if(strcmp(p_check->label, l) == 0) { exists = 1; break; }
    p_check = p_check->next;
  }
  if (exists) {
    printf("The label already exists. Duplicate can't be inserted.\n");
  } else {
    struct symbtab *p = (struct symbtab*)malloc(sizeof(struct symbtab));
    strcpy(p->label, l);
    printf("Enter the address: ");
    scanf("%d", &p->addr);
    p->next = NULL;
    if (size == 0) {
```

```
            first = p;
            last = p;
        } else {
            last->next = p;
            last = p;
        }
        size++;
    }
}
void display() {
    struct symbtab *p = first;
    if (size == 0) {
        printf("\nSymbol Table is empty.\n");
        return;
    }
    printf("\nLABEL\tADDRESS\n");
    while (p != NULL) {
        printf("%s\t%d\n", p->label, p->addr);
        p = p->next;
    }
}
void modify() {
    char l[10], nl[10];
    int add, choice;
    struct symbtab *p = first;
    printf("\n1. Modify Label\n2. Modify Address\n3. Modify Both\nEnter choice: ");
    scanf("%d", &choice);
    printf("Enter the existing label to modify: ");
    scanf("%s", l);
    int found = 0;
    while (p != NULL) {
        if (strcmp(p->label, l) == 0) {
            if (choice == 1 || choice == 3) {
                printf("Enter new label: ");
                scanf("%s", nl);
                strcpy(p->label, nl);
            }
            if (choice == 2 || choice == 3) {
                printf("Enter new address: ");
                scanf("%d", &add);
                p->addr = add;
            }
            found = 1;
            break;
```

```
        }
        p = p->next;
    }
    if(!found) printf("Label not found!\n");
}
void del() {
    char l[10];
    struct symbtab *p, *q;
    printf("Enter the label to be deleted: ");
    scanf("%s", l);
    if (size == 0) {
        printf("Table empty.\n");
        return;
    }
    p = first;
    if (strcmp(first->label, l) == 0) {
        first = first->next;
        free(p);
        if (first == NULL) last = NULL;
        size--;
    } else {
        q = p->next;
        int found = 0;
        while (q != NULL) {
            if (strcmp(q->label, l) == 0) {
                p->next = q->next;
                if (q == last) last = p;
                free(q);
                size--;
                found = 1;
                break;
            }
            p = p->next;
            q = q->next;
        }
        if(!found) printf("Label not found\n");
        else printf("Label deleted successfully.\n");
    }
}
```

Output:

Insert Operation:

```
--- SYMBOL TABLE IMPLEMENTATION ---
1. INSERT
2. DISPLAY
3. DELETE
4. SEARCH
5. MODIFY
6. END
Enter your option: 1
Enter the label: G
Enter the address: 21

LABEL    ADDRESS
G        21
```

Display Operation:

```
--- SYMBOL TABLE IMPLEMENTATION ---
1. INSERT
2. DISPLAY
3. DELETE
4. SEARCH
5. MODIFY
6. END
Enter your option: 2

LABEL    ADDRESS
G        21
A        1
B        2
```

Delete Operation:

```
--- SYMBOL TABLE IMPLEMENTATION ---
1. INSERT
2. DISPLAY
3. DELETE
4. SEARCH
5. MODIFY
6. END
Enter your option: 3
Enter the label to be deleted: A
Label deleted successfully.

LABEL   ADDRESS
G       21
B       2
```

Search Operation:

```
--- SYMBOL TABLE IMPLEMENTATION ---
1. INSERT
2. DISPLAY
3. DELETE
4. SEARCH
5. MODIFY
6. END
Enter your option: 4
Enter the label to be searched: G

Label 'G' found!
Address: 21
```

Modify Operation:

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, P

```
4. SEARCH
5. MODIFY
6. END
Enter your option: 5

1. Modify Label
2. Modify Address
3. Modify Both
Enter choice: 3
Enter the existing label to modify: B
Enter new label: #
Enter new address: 23

LABEL   ADDRESS
G       21
#       23
```